

Pre-Project Curriculum

Cross-Atlas Reproducibility of GWAS-Driven Cell-Type Prioritization in UC

Contents

Pre-Project Curriculum: Cross-Atlas Reproducibility of GWAS-Driven Cell-Type Prioritization in UC	3
Prerequisites and Self-Assessment	4
Module 0: Remediation (only if needed)	4
Module 1: GWAS Fundamentals	4
Lecture: What a GWAS measures	5
Lecture: Summary statistics format	6
Lecture: Polygenicity and the cell-type prioritization problem	6
Required readings	7
Optional readings	7
Exercises	7
Deliverable	8
Self-assessment checklist	8
Module 2: MAGMA and Gene-Level Analysis	8
Lecture: From SNPs to genes	8
Lecture: What a gene Z-score actually means	9
Lecture: The top-1000 cutoff	10
Required readings	10
Optional readings	10
Exercises	10
Deliverable	11
Self-assessment checklist	11
Module 3: Single-Cell RNA-seq	12
Lecture: What scRNA-seq measures	12
Lecture: The AnnData object	13
Lecture: Normalization and the count matrix	13
Lecture: Quality control	14
Lecture: Doublet detection and ambient RNA	14
Lecture: Cell-type annotation	14
Required readings	15
Optional readings	15
Exercises	15
Deliverable	16
Self-assessment checklist	16

Module 4: Cell-Type Prioritization Methods	16
Lecture: The conceptual framework	17
Lecture: scDRS in detail	17
Lecture: seismicGWAS in detail	18
Lecture: Why methods aren't independent	19
Required readings	19
Optional readings	19
Exercises	19
Deliverable	21
Self-assessment checklist	21
Module 5: Cell-Type Harmonization	21
Lecture: The harmonization problem	21
Lecture: The Cell Ontology	22
Lecture: scOntoMatch and the harmonization workflow	22
Lecture: Sub-clustering depth handling	23
Required readings	23
Optional readings	24
Exercises	24
Deliverable	24
Self-assessment checklist	24
Module 6: Statistical Methods for the Project	25
Lecture: Concordance metrics	25
Lecture: Bootstrap confidence intervals	26
Lecture: Multiple-testing correction	26
Lecture: Brown's method for combining correlated p-values	27
Lecture: Donor-level leave-one-out	27
Required readings	28
Optional readings	28
Exercises	28
Deliverable	29
Self-assessment checklist	29
Module 7: UC Biology	29
Lecture: Colon mucosa anatomy	29
Lecture: UC pathophysiology	30
Lecture: UC genetics	30
Lecture: Anti-integrin therapy and the Mennillo time-course atlas	31
Required readings	31
Optional readings	31
Exercises	31
Deliverable	32
Self-assessment checklist	32
Module 8: Atlas Integration and v2 Horizon	32
Lecture: The integration problem	32
Lecture: Integration methods	33
Lecture: Integration QC	33
Lecture: Why cross-tissue integration is hard	33
Required readings (optional for v1)	34
Optional readings	34

Exercises (optional)	34
Deliverable (optional)	34
Self-assessment checklist	34
Module 9: Reproducibility and Project Management	35
Lecture: Why reproducibility matters	35
Lecture: Git and GitHub for two-author projects	35
Lecture: Hummingbird (UCSC HPC) basics	36
Lecture: Project management for a 12-month analysis	36
Required readings	37
Optional readings	37
Exercises	37
Deliverable	38
Self-assessment checklist	38
Module 10: Project Orientation and M0 Readiness	38
Lecture: What “ready for M0” means	39
Comprehensive self-assessment	39
Pre-M0 questions for PI mentor	40
Final exercises before M0	40
Deliverable	41
Self-assessment checklist	41
Course Conclusion	41

Pre-Project Curriculum: Cross-Atlas Reproducibility of GWAS-Driven Cell-Type Prioritization in UC

A self-study course covering the science you need to execute the v1 plan. Designed for a UCSC junior in Bioinformatics with Python proficiency, basic statistics, and undergraduate-level biology, but no prior experience with GWAS analysis or single-cell methods.

The course is **project-aligned**: every exercise produces something you’ll actually use in M1, and the deliverables are the same artifacts the v1 plan asks for. Working through the curriculum is preparation for M0 lock-in, not separate work.

Pacing. 10 modules at ~10-15 hours each = 100-150 hours total. At full-time pace (40 hours/week), 3-4 weeks. At part-time pace alongside coursework (10 hours/week), 10-15 weeks. Recommended pace: 8-10 weeks at ~12 hours/week, finishing 2 weeks before the planned M0 lock-in.

How to use this document. Each module has the same structure: learning objectives, prerequisites, lecture content (substantial — these are not abstracts), required and optional readings, exercises with hints, a deliverable, and a self-assessment checklist. Work modules in order; some build on prior. Exercises are calibrated to take 2-4 hours each. If an exercise takes more than 6 hours you’re either stuck (ask the PI mentor) or going deeper than necessary (move on, come back later).

Prerequisites and Self-Assessment

Before starting Module 1, confirm you can comfortably do the following. If any answer is “no” or “not really,” do the remediation in Module 0 before proceeding.

Python. Read and write Python for data manipulation. Familiar with pandas DataFrames, numpy arrays, basic plotting (matplotlib or seaborn), file I/O. Can install packages with pip and manage dependencies.

Statistics. Comfortable with: probability, hypothesis testing, p-values, multiple testing correction (Bonferroni at minimum, FDR is a plus), correlation (Pearson and Spearman), basic regression. Comfortable doesn’t mean expert — Module 6 will build the more advanced methods.

Biology fundamentals. Understand: DNA → RNA → protein, what a gene is, what an allele is, what a SNP is, what gene expression means, what a cell type is at a basic level. Don’t need to know cell biology in depth — Module 7 covers the disease-specific biology.

Linear algebra. Comfortable with vectors, matrices, matrix multiplication, dimensionality reduction concepts (PCA at a high level). Don’t need to know how PCA is computed; just need to understand “PCA reduces high-dimensional data to lower dimensions for visualization and analysis.”

Tools. Familiar with: command line (bash), git basics (clone, commit, push, pull), reading documentation. SSH into a remote server is a plus but can be learned in Module 9.

Module 0: Remediation (only if needed)

If your Python is rusty, work through the *Python for Data Analysis* book (Wes McKinney) chapters 5–10, ~10 hours. If your statistics are rusty, work through *StatQuest* YouTube playlist on hypothesis testing and confidence intervals, ~5 hours. If your biology is rusty, watch *Khan Academy* genetics unit, ~5 hours. Combined: at most 1 week to remediate gaps.

Module 1: GWAS Fundamentals

Learning objectives. Explain what a GWAS measures and what summary statistics look like. Identify the columns in a GWAS summary statistics file. Compute genomic inflation factor (λ_{GC}). Recognize ancestry-related issues with linkage disequilibrium reference panels. Understand why the MHC region is special for immune diseases.

Time estimate. 12–15 hours.

Prerequisites. Module 0 if needed.

Lecture: What a GWAS measures

A genome-wide association study (GWAS) tests, for each of millions of common SNPs across the genome, whether one allele associates with a trait or disease more frequently than the other. The output is a per-SNP test statistic, p-value, and effect size estimate.

The fundamental data structure of a GWAS is the *case-control comparison*. You have, say, 12,000 people with ulcerative colitis and 33,000 controls. For each of ~10 million SNPs, you ask: “Does the frequency of allele A differ between cases and controls more than expected by chance?” The answer is summarized as a Z-score (effect size divided by standard error) and a p-value.

A few facts that drive everything downstream:

Most signal is sub-significant but real. The genome-wide significance threshold is $p < 5 \times 10^{-8}$, accounting for ~1 million independent tests across the genome. UC has hundreds of genome-wide-significant loci. But polygenic traits like UC have *thousands* of variants each contributing tiny effects below the significance threshold. Modern cell-type prioritization methods use the full distribution of GWAS Z-scores, not just the genome-wide-significant hits, because the sub-threshold signal is informative.

SNPs are correlated. Variants don’t sit alone in the genome — they inherit as part of haplotypes with their physical neighbors. This correlation, called *linkage disequilibrium* (LD), is the central statistical complication in GWAS. If a true causal variant is in LD with 50 nearby SNPs, all 51 will appear genome-wide-significant in the GWAS, and you can’t tell which is causal without further work. LD also means the “1 million independent tests” approximation isn’t tight — the effective number of independent tests in the human genome is closer to 1 million for European populations, but varies by ancestry.

Ancestry matters everywhere. LD patterns differ between populations: Europeans, East Asians, South Asians, and African populations all have somewhat different haplotype structure. A GWAS done in Europeans may have different signal at a given locus than the same trait studied in East Asians, because the tag SNPs differ. When you do downstream analysis (like MAGMA), you need to use an LD reference panel matched to the GWAS ancestry. For multi-ancestry GWAS, this gets complicated — most analyses use European reference as an approximation, which we acknowledge in DECISIONS.md.

The MHC region is special. Chromosome 6, roughly 28.5–33.5 Mb (in GRCh37 coordinates), contains the major histocompatibility complex (HLA genes). This region has extreme LD — single variants can tag dozens of correlated SNPs — and contains the most variable genes in the human genome. For autoimmune diseases like UC, the MHC region typically has the strongest GWAS signal. But because of the complex LD structure, including MHC unfiltered in downstream analysis can produce misleading results. Standard practice in cell-type prioritization is to either exclude MHC entirely from the gene set or report a sensitivity analysis. Our v1 plan excludes MHC.

Lecture: Summary statistics format

A GWAS summary statistics file is tab-separated, with one row per tested SNP. The columns vary by GWAS but typically include:

SNP	CHR	BP	A1	A2	FRQ_A1	BETA	SE	P	N
rs1234567	1	100123	A	G	0.234	-0.0421	0.0089	2.31e-06	62000

SNP is the variant ID, usually an rsID. **CHR** and **BP** are chromosome and base-pair position; you must know which genome build (GRCh37 / hg19 vs GRCh38 / hg38) the coordinates are in, because they're not the same. Most GWAS published before 2020 use GRCh37; many newer GWAS use GRCh38. **A1** is the effect allele (the allele whose effect size BETA refers to); **A2** is the other allele. Sign of BETA depends on which allele you call the effect allele — flipping this sign is a common mistake that produces nonsensical results. **FRQ_A1** is the frequency of A1 in the cohort (or, sometimes, in a reference panel like 1000 Genomes). **BETA** is the per-allele effect size on the (typically log-odds) scale; **SE** is its standard error. **P** is the p-value. **N** is sample size — sometimes per-SNP (different SNPs have different N if some are imputed and some aren't), sometimes a fixed total.

Quality control on a downloaded GWAS file checks several things:

1. **Genome build.** Verify the coordinates match the build you'll use downstream.
2. **Allele consistency.** Check whether A1 is consistently the minor allele or the effect allele; different GWAS use different conventions.
3. **Imputation quality.** Many GWAS report INFO scores for imputation; drop SNPs with INFO < 0.6.
4. **Allele frequency.** Drop SNPs with MAF < 0.01 (rare variants are not well-powered for common-variant GWAS).
5. **Genomic inflation factor (λ_{GC}).** This is the median chi-squared statistic divided by the expected median under the null (0.456 for one degree of freedom). $\lambda_{GC} \approx 1$ means the GWAS is well-controlled for population stratification; $\lambda_{GC} > 1.1$ is a red flag suggesting uncontrolled confounding. UC GWAS typically have λ_{GC} around 1.05–1.10.

Lecture: Polygenicity and the cell-type prioritization problem

UC is a *highly polygenic* disease. Hundreds of genome-wide-significant loci, thousands of sub-threshold contributing variants. The biological question becomes: "Across all these tiny effects, which cell types in the colon mucosa carry the genetic signal?"

This is what scDRS, seismicGWAS, and related methods answer. They take the full distribution of gene-level GWAS Z-scores (computed by MAGMA from the SNP-level summary statistics — Module 2) and combine it with single-cell expression data to nominate disease-relevant cell types. The intuition: if T cells preferentially express genes that are also UC GWAS hits, T cells are probably implicated in UC pathogenesis.

This intuition works because of the *omnigenic model* (Boyle, Li, Pritchard 2017, *Cell*). Common-variant disease genetics generally implicates many genes, most of which are not "core" disease genes but instead "peripheral" — they affect the regulation,

expression, or function of core genes in disease-relevant cell types. The cell types where the peripheral genes are expressed are the cell types where disease biology lives.

Required readings

de Lange et al. 2017. *Genome-wide association study implicates immune activation of multiple integrin genes in inflammatory bowel disease.* Nat Genet 49:256–261. Read sections: Abstract, Introduction, Results (focus on UC results), Discussion. Skim Methods. ~2 hours.

Visscher et al. 2017. *10 Years of GWAS Discovery: Biology, Function, and Translation.* Am J Hum Genet 101:5–22. A canonical retrospective on GWAS methodology. Read fully. ~2 hours.

Boyle, Li, Pritchard 2017. *An Expanded View of Complex Traits: From Polygenic to Omnigenic.* Cell 169:1177–1186. Read fully. ~1.5 hours.

Optional readings

Liu et al. 2023. Multi-ancestry IBD meta-analysis. Useful for understanding the cross-GWAS sensitivity comparison.

Yengo et al. 2022. Height GWAS — useful background for the positive control.

Exercises

Exercise 1.1: Download and inspect de Lange UC GWAS. Find de Lange 2017 on GWAS Catalog (GCST004131). Download the summary statistics. Inspect the columns and answer: - What genome build are the coordinates in? - What does each column mean? - How many SNPs are in the file? - What's the smallest p-value? - Which SNP has the smallest p-value? Which gene is it near (use UCSC Genome Browser or GWAS Catalog locuszoom view)?

Exercise 1.2: Compute λ_{GC} . Write a Python script that reads the de Lange summary statistics file, computes the chi-squared statistic for each SNP ($= (BETA/SE)^2$), and reports the median chi-squared divided by 0.456. Compare your result to the published λ_{GC} if reported.

Hint: `pandas.read_csv` with `sep='\t'` should work. You'll need to filter out rows with missing values. The expected answer for de Lange UC is $\lambda_{GC} \approx 1.05$ – 1.10 .

Exercise 1.3: Count genome-wide-significant SNPs. From the same file, count how many SNPs have $p < 5 \times 10^{-8}$. Then count how many *independent* loci this represents — define a “locus” as any 1-Mb window containing at least one significant SNP. The published de Lange paper reports ~200 loci across IBD; you should get a similar order-of-magnitude.

Exercise 1.4: MHC inspection. Filter the de Lange file to chr 6: 28,477,797–33,448,354 (GRCh37 coordinates). How many SNPs are in this region? What fraction

of them are genome-wide significant? Compare to a non-MHC region of similar size on chr 6.

Deliverable

A Jupyter notebook (`module1_gwas_fundamentals.ipynb`) containing all four exercises, with a final markdown cell summarizing:

1. Top 10 UC-associated SNPs from de Lange (rsID, chromosome, position, p-value, nearest gene)
2. λ_{GC} for de Lange
3. Number of independent loci (1-Mb-window definition)
4. MHC region SNP density and significance fraction

This notebook becomes the foundation for M1 GWAS download work — keep it.

Self-assessment checklist

- ☐ I can explain what a GWAS measures in 2 sentences without using the word “genome-wide.”
 - ☐ I can explain why LD complicates GWAS interpretation.
 - ☐ I know what columns are in a GWAS summary statistics file.
 - ☐ I can compute λ_{GC} .
 - ☐ I understand why MHC is excluded from cell-type prioritization gene sets.
 - ☐ I have a Jupyter notebook with the de Lange GWAS loaded and inspected.
-

Module 2: MAGMA and Gene-Level Analysis

Learning objectives. Explain how MAGMA converts SNP-level statistics to gene-level statistics. Run MAGMA end-to-end on a small dataset. Generate a top-1000-gene .gs file for scDRS. Understand the role of the LD reference panel.

Time estimate. 10–12 hours.

Prerequisites. Module 1.

Lecture: From SNPs to genes

scDRS and seismicGWAS don’t take SNP-level data as input — they take *gene-level* statistics. The standard tool for converting one to the other is **MAGMA** (de Leeuw et al. 2015, *PLOS Comp Biol*).

MAGMA’s gene-based test asks: “For each gene in the genome, given all the SNPs in or near that gene and their LD structure, is there more association signal than expected by chance?” The output is a per-gene Z-statistic and p-value.

The procedure has three steps:

Step 1: SNP-to-gene annotation. Each SNP is assigned to one or more genes if it falls within a window around the gene’s transcribed region. The convention is 10 kb

upstream and 10 kb downstream, though some analyses use larger windows (35 kb, 50 kb) to capture distal regulatory elements. The trade-off: larger windows capture more distal SNPs but produce more SNP-to-gene multiple assignment, which dilutes the per-gene test statistic. Our plan uses the conventional 10 kb / 10 kb.

Step 2: Gene-based test. For each gene with SNPs assigned to it, MAGMA combines the SNP-level Z-statistics into a gene-level Z-statistic, accounting for the LD structure between the SNPs. The test is a sum-of-squared-Z-statistics, with a correlation correction estimated from a reference LD panel. The correlation correction matters: SNPs in high LD effectively contribute less independent information than SNPs in low LD.

The *LD reference panel* is critical. It must match the ancestry of the GWAS. For European-ancestry GWAS, use the 1000 Genomes Phase 3 EUR panel. For multi-ancestry GWAS, a clean solution requires per-ancestry MAGMA runs followed by meta-analysis — that’s a research project, not a standard pipeline. As a practical first-pass approximation, use 1000G EUR for multi-ancestry GWAS and document the approximation. Our plan does this for Liu 2023.

Step 3: Conversion to downstream format. The MAGMA output is a `.genes.out` file with per-gene Z-scores and p-values. To use these with scDRS, you select the top N genes (typically 1000) by Z-score and write them to a `.gs` file in scDRS’s expected format: one line per trait, listing gene names with their Z-scores as weights. For seismicGWAS, you skip the `.gs` step and provide the full Z-score table directly.

Lecture: What a gene Z-score actually means

The MAGMA gene Z-score is *not* a measure of “how much this gene causes the disease.” It’s a measure of “how much association signal is concentrated in or near this gene’s transcribed region, after accounting for SNP density and LD.”

Several caveats follow from this:

Long genes accumulate more SNPs. A 200-kb gene has more SNPs than a 20-kb gene, so even under the null, longer genes have somewhat higher gene-level test statistics. MAGMA partially controls for this in the gene-based test, but residual gene-length bias is one reason seismicGWAS includes gene length as a confounder.

Genes near GWAS hits inherit signal. If a strong GWAS hit lies in the intergenic region between two genes, both genes (within 10 kb) inherit the signal. The downstream cell-type analysis can’t distinguish which gene is the actual mediator.

The gene Z-score is *not* causal. A high Z-score for IL23R doesn’t mean IL23R is “the cause” of UC. It means the GWAS signal in or near IL23R is strong. Causal interpretation requires fine-mapping (a different methodology) and ideally functional follow-up.

For cell-type prioritization, none of this matters as long as you remember the question being asked: “Which cell types preferentially express genes near UC GWAS signal?” The downstream conclusion is about cell types, not genes.

Lecture: The top-1000 cutoff

scDRS uses a *gene set* as the disease representation: the top N MAGMA genes by Z-score, with their Z-scores as weights. The standard choice is $N = 1000$ (scDRS's default). Why 1000?

The matched-null Monte Carlo procedure that scDRS uses needs a fixed-size gene set; control gene sets are sampled to have the same size. Larger gene sets ($N = 2000, 5000$) include more sub-threshold signal but also more noise. Smaller sets ($N = 200, 500$) are more concentrated on top hits but lose polygenicity. Empirically, 1000 is a reasonable compromise that captures most of the polygenic signal without too much noise.

Our plan locks in $N = 1000$ for v1 and notes that sensitivity to this cutoff isn't formally tested. Reviewers may ask; the response is "we used the scDRS default."

Required readings

de Leeuw et al. 2015. *MAGMA: Generalized Gene-Set Analysis of GWAS Data*. PLOS Comp Biol 11:e1004219. Read sections 1–3 fully; skim section 4. ~2 hours.

MAGMA documentation. <https://ctg.cncr.nl/software/magma> — read the manual, especially the sections on gene annotation and gene-based test. ~1.5 hours.

Optional readings

Watanabe et al. 2017. FUMA. Tool that wraps MAGMA in a web interface. Useful background.

Skene et al. 2018. EWCE. Specificity-based cell-type prioritization (precursor to seismicGWAS). Read for context.

Exercises

Exercise 2.1: Install MAGMA. Download MAGMA from <https://ctg.cncr.nl/software/magma>. Install on your local machine. Run the example tutorial that ships with MAGMA. Verify outputs match the expected outputs in the tutorial.

Hint: MAGMA is a precompiled C++ binary. No installation beyond extracting and adding to your PATH. The tutorial uses small example files; should run in <5 minutes.

Exercise 2.2: Download 1000G EUR LD reference panel. From the MAGMA website, download the 1000 Genomes Phase 3 European reference panel in PLINK binary format (.bed/.bim/.fam). This is ~2 GB — make sure you have disk space. Verify the file integrity (md5 if provided).

Exercise 2.3: Run MAGMA on de Lange UC GWAS. Three-step pipeline:

```
# Step 1: SNP-to-gene annotation
magma --annotate window=10,10 \
      --snp-loc snps.loc \
      --gene-loc NCBI37.3.gene.loc \
```

```

--out step1_annot

# Step 2: Gene-based test
magma --bfile g1000_eur \
--pval gwas_uc_delange.pval N=45975 \
--gene-annot step1_annot.genes.annot \
--out step2_gene

```

You'll need to convert the de Lange summary statistics to MAGMA's expected format: a .snploc file (chromosome, position) and a .pval file (SNP, p-value). The MAGMA manual covers the formats.

After running, inspect step2_gene.genes.out. The top genes by Z-score should include IL23R, JAK2, TYK2, NKX2-3, IL12B — the canonical IBD-associated genes. If they're not there, debug.

Exercise 2.4: Generate top-1000-gene .gs file for scDRS. Write a Python script that reads step2_gene.genes.out, sorts by Z-score, takes the top 1000 genes, **excludes any genes in the MHC region** (chr 6: 28,477,797-33,448,354 in GRCh37), and writes them in scDRS's .gs file format:

```

TRAIT    GENESET
UC_delange  IL23R:8.42,JAK2:7.21,TYK2:6.85,...

```

The scDRS documentation specifies the exact format; verify yours matches.

Hint: You'll need a gene-to-chromosome lookup. The NCBI37.3.gene.loc file has this. MHC genes can be identified by (chr == 6) & (start < 33,448,354) & (end > 28,477,797).

Exercise 2.5: Sanity-check top genes. Open the top-50 genes in your .gs file. For each, search PubMed or the GWAS Catalog: is this gene known to be associated with UC or IBD? You should find ~80%+ have direct literature support. The remaining 20% may be novel or near-significant; investigate one or two.

Deliverable

A directory (module2_magma/) containing: - step1_annot.genes.annot and step2_gene.genes.out from MAGMA - uc_delange_top1000.gs (the MHC-excluded scDRS gene set file) - module2_summary.md with a markdown table of top-30 genes, their Z-scores, and a literature note (UC-associated / IBD-associated / unknown)

This deliverable becomes the M1 starting point. You'll repeat for Liu 2023 and Trubetskoy 2022 schizophrenia in M1.

Self-assessment checklist

- ☐ I understand the three steps of MAGMA: SNP annotation, gene-based test, downstream conversion.
- ☐ I understand why the LD reference panel must match GWAS ancestry.
- ☐ I can explain what the gene-level Z-score does and doesn't mean.

- ☐ I have run MAGMA end-to-end on de Lange.
 - ☐ I have a top-1000-gene MHC-excluded .gs file ready for scDRS.
-

Module 3: Single-Cell RNA-seq

Learning objectives. Explain the structure of an scRNA-seq dataset and the AnnData object. Load a published atlas, compute QC metrics, generate UMAPs, and visualize cell-type annotations. Understand normalization, doublet detection, and ambient RNA correction at a conceptual level. Recognize the differences between published preprocessing protocols and why they matter for cross-atlas comparison.

Time estimate. 15–18 hours.

Prerequisites. Module 1, Module 2.

Lecture: What scRNA-seq measures

Single-cell RNA sequencing measures gene expression in individual cells. The output, after extensive computational processing, is a **count matrix**: one row per cell, one column per gene, each entry a non-negative integer counting the number of mRNA molecules from that gene that were detected in that cell.

Modern scRNA-seq atlases routinely measure 100,000 to 1,000,000+ cells per study. Each cell expresses ~3,000–8,000 genes detectably above background. The technology has limitations:

Sparsity. Most entries in the count matrix are zero. A typical cell expresses ~5,000 genes out of the ~20,000 protein-coding human genes; the rest read as zero, but these “zeros” are a mix of genuine non-expression and technical dropout (the gene was expressed but the molecule wasn’t captured). This sparsity is the single biggest computational challenge in scRNA-seq analysis.

Counts vary per cell. Different cells have different total counts (UMIs) — typically ranging from ~500 to ~50,000 per cell. Higher-count cells are not necessarily “better” cells; some cell types simply express more total mRNA. But the variation makes raw counts not directly comparable across cells, motivating normalization.

Doublets and noise. scRNA-seq protocols sometimes capture two cells in the same droplet, producing artifactual “cells” with mixed expression profiles (doublets). Cell-free RNA in the buffer contaminates each droplet (ambient RNA). Both inflate the apparent expression of certain genes in cells where they’re not actually expressed.

Cell-type identity is inferred, not measured. A cell doesn’t come with a label. Cell-type identity is *inferred* by clustering cells in a high-dimensional gene-expression space, then labeling clusters based on marker gene expression. This inference is subjective — different analysts can arrive at different cell-type labels for the same data.

Lecture: The AnnData object

The standard data structure for scRNA-seq in Python is AnnData from the scanpy ecosystem. An AnnData object has the following structure:

AnnData	
├ X	cell × gene count matrix (sparse, scipy CSR)
├ obs	DataFrame with one row per cell (cell metadata)
├ var	DataFrame with one row per gene (gene metadata)
├ obsm	multidimensional cell-level annotations (PCA, UMAP coords)
├ varm	multidimensional gene-level annotations
├ layers	alternative count matrices (raw, normalized, denoised)
└ uns	unstructured metadata (cluster info, color schemes)

The X matrix is the core data. Different layers can hold raw counts, log-normalized counts, scaled counts, etc. **scDRS expects raw counts in X (or in a specified layer).** seismicGWAS expects log-normalized counts. If you use the wrong layer, results are silently wrong.

The obs DataFrame holds per-cell metadata: cell type label, donor ID, disease status, total counts per cell, percentage of mitochondrial reads, etc. Our v1 plan adds donor-attribution metadata (originating_study, tissue, disease_status, assay_protocol) to obs.

The var DataFrame holds per-gene metadata: gene symbol, Ensembl ID, gene type (protein-coding, lncRNA, etc.). Critical for our work: gene symbols change over time as HGNC updates the official symbol list. Different atlases may use different symbol versions, requiring harmonization.

Lecture: Normalization and the count matrix

Raw counts can't be directly compared across cells because total counts vary. Normalization addresses this by dividing each cell's counts by a per-cell scaling factor, then optionally log-transforming.

The simplest normalization is *library-size normalization*: divide each cell's counts by total counts and multiply by 10,000 (CP10K — counts per 10,000). Then take $\log(x + 1)$. Result: a matrix where each cell's values represent expression relative to that cell's total mRNA, log-scaled.

More sophisticated normalizations (scrn pooling, SCTransform variance stabilization) produce somewhat different matrices. For our cross-atlas comparison, *the published atlases used different normalizations*, and we don't re-normalize uniformly — we accept this heterogeneity per the §2.3 atlas-preprocessing policy. The marker-gene QC at end of M2 catches the worst cross-atlas inconsistencies.

For scDRS specifically: the method requires raw counts and does its own normalization internally, controlled by covariates. So normalization differences across atlases matter less for scDRS than for other tools. For seismicGWAS: the specificity score uses log-normalized expression and does normalize within atlas, but cross-atlas specificity comparisons depend on the within-atlas normalization being reasonable.

Lecture: Quality control

scRNA-seq QC removes low-quality cells (likely dying or empty droplets) and ambient/doublet artifacts. Standard QC filters:

Minimum gene count per cell. Cells expressing fewer than ~200 genes are likely dying or ambient. Threshold varies by tissue: gut mucosa typically uses 200–500.

Maximum gene/UMI count per cell. Very high counts may indicate doublets. Threshold varies.

Mitochondrial percentage. High % mitochondrial reads (>20–25%) indicates dying cells releasing cytoplasmic mRNA. Threshold depends on tissue.

Ribosomal percentage. Some atlases also filter on % ribosomal reads.

These filters are applied by the original study authors. Our v1 plan does not re-apply them uniformly — we trust each atlas’s published QC. This is a deliberate choice, documented in DECISIONS.md and disclosed as a Limitation.

Lecture: Doublet detection and ambient RNA

Two technical artifacts deserve special attention:

Doublets. When two cells are captured in the same droplet, the resulting count vector is a mixture of two cell-type expression profiles. Methods like Scrublet (Wolock 2019) and DoubletFinder (McGinnis 2019) detect doublets by simulating mixed profiles and flagging cells that look similar to the simulations. Different atlases use different doublet detection tools with different thresholds. Cells flagged as doublets are typically removed.

Ambient RNA. Cell-free RNA in the buffer contaminates each droplet at low levels. Tools like SoupX (Young 2020) and CellBender (Fleming 2023) estimate the ambient RNA profile and subtract it from each cell. Some atlases apply this; others don’t.

For cross-atlas comparison, these preprocessing differences are a real source of confounding. We accept them per the v1 policy and disclose in Limitations.

Lecture: Cell-type annotation

After QC, the standard scRNA-seq pipeline runs:

1. Normalization (typically log1p of CP10K)
2. Highly variable gene (HVG) selection (top ~2000 most variable genes)
3. PCA on HVGs (typically 30–50 components)
4. Nearest-neighbor graph construction in PCA space
5. Clustering (Leiden or Louvain algorithm)
6. UMAP for visualization
7. Cell-type annotation: identify each cluster’s marker genes and assign a cell-type label

The cell-type annotation is the most subjective step. Two analysts can produce different cluster numbers and different labels from the same data. Our v1 plan accepts each

atlas's published cell-type labels and harmonizes across atlases using scOntoMatch + manual crosswalk + marker-gene confirmation.

Required readings

Luecken & Theis 2019. *Current best practices in single-cell RNA-seq analysis: a tutorial.* Mol Syst Biol 15:e8746. Read sections 1-4 fully. ~3 hours. This is the canonical scRNA-seq methods reference.

scanpy tutorials. <https://scanpy.readthedocs.io/en/stable/tutorials/basics/clustering.html> — work through the basic clustering tutorial end-to-end. ~3 hours.

Smillie et al. 2019. *Intra- and Inter-cellular Rewiring of the Human Colon during Ulcerative Colitis.* Cell 178:714-730. Read Abstract, Introduction, Results focusing on cell-type findings, Methods. ~2 hours. This is one of our three core UC atlases.

Optional readings

Heumos et al. 2023. *Best practices for single-cell analysis across modalities.* Nat Rev Genet — updated version of Luecken & Theis. Useful for newer methods.

Wolock et al. 2019 (Scrublet) and **Young et al. 2020** (SoupX) for doublet/ambient details.

Exercises

Exercise 3.1: Set up scanpy environment. Install scanpy and dependencies in a fresh conda environment:

```
conda create -n uc-scrna python=3.10
conda activate uc-scrna
pip install scanpy anndata pandas numpy scipy matplotlib seaborn leidenalg python-igraph
```

Verify with `python -c "import scanpy; print(scanpy.__version__)"`.

Exercise 3.2: Download Smillie 2019 from Single Cell Portal SCP259. Note: this requires a Single Cell Portal account (free). Download the colon mucosa AnnData file. ~1-2 GB. Inspect: - Number of cells, genes - Available obs columns (look for cell_type, donor_id, disease_status) - Available layers (raw counts? log-normalized?) - Cell-type composition (how many T cells, B cells, etc.)

Exercise 3.3: QC and visualization. Run scanpy's standard pipeline on the Smillie atlas: 1. Compute QC metrics (n_genes, n_counts, % mito). 2. Visualize QC metrics as violin plots — do you see any outlier populations? 3. The data should already be QC-filtered by Smillie 2019. Confirm reasonable per-cell distributions. 4. Compute PCA, neighbors, UMAP. 5. Plot UMAP colored by cell_type, donor_id, disease_status.

If your UMAP is dramatically different from the published Smillie 2019 figures, something is wrong. Most likely: you're running on a different gene set or normalization than the published one. Don't worry about reproducing the figure exactly; just confirm cell types cluster reasonably.

Exercise 3.4: Marker-gene inspection. For each broad cell type in the Smillie atlas: - T cell: check CD3D, CD3E, CD4, CD8A - B cell: check CD19, MS4A1 - Plasma: check MZB1, JCHAIN - Enterocyte: check VIL1, FABP1 - Goblet: check MUC2, TFF3 - Monocyte/Macrophage: check CD68, CD14 - Mast: check TPSB2, KIT

Use `sc.pl.dotplot` to visualize marker gene expression by cell type. Confirm each cell type's canonical markers are expressed in the expected cluster.

If a cell type's markers are *not* expressed where expected, the cell-type labels in this atlas may be problematic. (Unlikely for Smillie, but worth checking.)

Exercise 3.5: Generate the v1 covariate file. Create `smillie_covariates.tsv` with columns: - `cell_id` (matching `adata.obs_names`) - `log_n_genes` - `log_n_counts` - `donor` (one-hot encoded if categorical) - `sample` (one-hot encoded if available) - `sex` (one-hot encoded if metadata exists; otherwise document and skip)

Write to TSV format. This is the actual M1 deliverable for Smillie's covariate file — keep it.

Deliverable

A directory (`module3_scrna/`) containing: - A Jupyter notebook with the Smillie pipeline run end-to-end - UMAP plots colored by cell type, donor, disease status (PNG or PDF) - Marker-gene dotplot - `smillie_covariates.tsv` ready for scDRS

Self-assessment checklist

- ☐ I understand what an `AnnData` object is and how to navigate one.
 - ☐ I can compute QC metrics and identify low-quality cells.
 - ☐ I can run scanpy's standard pipeline (normalize, PCA, neighbors, UMAP, clustering).
 - ☐ I understand the difference between raw counts (scDRS input) and log-normalized counts (seismicGWAS input).
 - ☐ I have generated a covariate file ready for scDRS.
-

Module 4: Cell-Type Prioritization Methods

Learning objectives. Explain the conceptual framework of polygenic-disease cell-type prioritization. Describe how scDRS computes cell-level scores and tests cell-type enrichment. Describe how seismicGWAS computes specificity scores and runs cell-type-trait regression. Understand why these methods aren't statistically independent. Run both methods on tutorial data and inspect outputs.

Time estimate. 15-18 hours.

Prerequisites. Modules 1, 2, 3.

Lecture: The conceptual framework

The question all these methods answer: “Given a GWAS for trait T and a single-cell atlas of tissue X, which cell types in X carry the genetic signal for T?”

The general approach: combine gene-level GWAS Z-scores (from MAGMA) with single-cell expression to compute a per-cell or per-cell-type “disease relevance score.” Cells or cell types with high scores are nominated as disease-relevant.

The methods differ in how they compute this combination:

scDRS (Zhang et al. 2022) computes a per-cell score: each cell’s expression of GWAS-implicated genes, normalized against a Monte Carlo null distribution sampled from the same atlas. Cell types are then tested for elevated mean scores via a group analysis.

seismicGWAS (Lai et al. 2025) computes a per-cell-type *specificity score* per gene: how specifically each gene is expressed in each cell type relative to the rest of the atlas. Then it regresses gene-level GWAS Z-scores against specificity scores: cell types whose specificity-weighted Z-scores are high are nominated as disease-relevant.

MAGMA gene-property regression (Watanabe et al. 2019) is the simplest: for each cell type, compute average gene expression per cell type, regress against MAGMA Z-scores, and test the regression coefficient. This is the methodological precursor to seismicGWAS.

Other methods (sc-linker, scPagwas, scGWAS) exist but aren’t in our v1.

Lecture: scDRS in detail

scDRS (single-cell disease relevance score) is documented in Zhang et al. 2022 *Nat Genet* 54:1572–1580.

Step 1: Per-cell raw score. For each cell c and the disease gene set G (top 1000 MAGMA genes with Z-scores as weights):

$$\text{raw_score}(c) = \sum_g [w_g \times \text{normalized_expression}(c, g)]$$

where w_g is the Z-score of gene g in G and $\text{normalized_expression}(c, g)$ is g ’s expression in cell c after standard normalization.

Step 2: Monte Carlo null. scDRS samples 1000 *control gene sets*, each matched to G on size and on per-gene mean expression and variance. For each control set, compute the per-cell raw score. Result: per cell, 1000 null raw scores.

Step 3: Per-cell Z-score. For each cell:

$$\text{score}(c) = (\text{raw_score}(c) - \text{mean of null raw scores for } c) / \text{sd of null raw scores for } c$$

This is the per-cell scDRS Z-score. Higher Z-scores = the cell preferentially expresses GWAS-implicated genes relative to expression-matched controls.

Step 4: Group analysis. For a cell-type partition (e.g., broad cell types T, B, plasma, etc.), test whether each cell type has elevated mean Z-scores relative to the rest of

the atlas. The test uses the same Monte Carlo null distributed over cell types. Output: per-cell-type p-value.

Key statistical caveat. The Monte Carlo null is *atlas-specific*. The same biology can produce different per-cell-type p-values in two atlases simply because the cell-type composition differs (different cell types contribute to the null). This is why our v1 plan uses *Z-scores* (not p-values) for cross-atlas concordance metrics.

Covariates. scDRS regresses out per-cell covariates (`log_n_genes`, `log_n_counts`, donor, sample, sex if available) before scoring. This controls for technical and demographic confounders.

Computational cost. ~30–45 minutes per (atlas, GWAS) for the compute-score step on a 250k-cell atlas with 15–20 GB RAM. Group analysis is post-processing on the per-cell scores and runs in seconds.

Lecture: seismicGWAS in detail

seismicGWAS (specificity-based cell-type-trait association) is documented in Lai et al. 2025 *Nat Commun* 16:8744.

Step 1: Specificity score per gene per cell type. For each gene g in each cell type c :

$\text{specificity}(g, c) = \text{avg_expression}(g \text{ in cells of type } c) / \text{global_avg_expression}(g)$

with a variance-stabilizing transform applied. The intuition: a gene is “specific to” a cell type if it’s expressed there but not elsewhere. T-cell-receptor genes are highly specific to T cells; housekeeping genes (GAPDH) are not specific to any cell type.

The specificity score is *robust* — re-clustering the atlas at slightly different resolutions doesn’t drastically change specificity scores. This is one reason seismicGWAS is more reproducible across runs than methods that depend on cluster identity.

Step 2: Regression. For each cell type c , regress gene-level GWAS Z-scores on specificity scores:

$Z_g \sim \beta_0 + \beta_c \times \text{specificity}(g, c) + \gamma \times \text{covariates} + \epsilon$

Covariates include gene length, gene-gene LD score, transcript count (controlling for MAGMA’s gene-length bias and other confounders).

The cell type’s *trait association coefficient* β_c is the headline output: how strongly the cell type’s specificity profile correlates with the GWAS Z-score profile.

Step 3: Significance test. Standard regression p-value on β_c , optionally with permutation-based null. The output is per-cell-type association strength and p-value.

Computational cost. ~5–15 minutes per regression. Specificity scores are GWAS-independent — compute once per atlas, reuse across GWAS.

Cross-method comparison. seismicGWAS β_c and scDRS group-analysis Z are conceptually similar measures (both quantify “how strongly does this cell type associate with the trait”), but they’re computed differently and aren’t statistically independent — both consume MAGMA gene Z-scores.

Lecture: Why methods aren't independent

A common pitfall in cross-method comparisons: assuming agreement between methods means the result is robust. Not necessarily.

scDRS, seismicGWAS, and MAGMA gene-property all share the same upstream input: gene-level Z-scores from MAGMA. If two methods strongly agree, it could be because: 1. The biology is real and both methods detect it (the desired interpretation), OR 2. Both methods are dominated by the same upstream signal-to-noise pattern (mechanical agreement)

The methods differ in how they aggregate gene-level signal to cell-type-level signal:
- scDRS uses gene-set membership (binary top-1000 cutoff) + Monte Carlo null at the cell level
- seismicGWAS uses continuous gene Z-scores + cell-type specificity weighting
- MAGMA gene-property uses average expression per cell type + regression

Cross-method *disagreement* is more informative than agreement: if scDRS says “T cells are UC-relevant” but seismicGWAS says they aren’t, the methods are picking up different aspects of the data. Investigation reveals which.

For our v1, we report cross-method concordance as a sensitivity check, not as evidence of robustness. The paper acknowledges shared MAGMA inputs in Methods.

Required readings

Zhang et al. 2022. *Polygenic enrichment distinguishes disease associations of individual cells in single-cell RNA-seq data.* Nat Genet 54:1572–1580. Read Abstract, Introduction, Results, Methods sections on scoring and group analysis. ~3 hours.

Lai et al. 2025. *seismicGWAS: a robust framework for trait-cell type association.* Nat Commun 16:8744. Read fully. ~2 hours.

Skene et al. 2018. *Genetic identification of brain cell types underlying schizophrenia.* Nat Genet 50:825–833. Read for the EWCE specificity-based methodology that seismicGWAS extends. ~2 hours.

Bryois et al. 2020. *Genetic identification of cell types underlying brain complex traits.* Nat Genet 52:482–493. Read for the cross-atlas methodology that our v1 paper extends to UC. ~2 hours.

Optional readings

Watanabe et al. 2019. MAGMA gene-property original application across 43 atlases × 26 traits. The methodological precursor to scDRS for population-scale analyses.

Lakkis et al. 2024. Single-atlas UC application of scDRS, scPagwas, scGWAS.

Exercises

Exercise 4.1: Install scDRS. In your conda environment:

```
pip install scdrs==1.0.4
```

Verify the version installed and the CLI works: `scdrs --help`.

Exercise 4.2: Run scDRS tutorial. Follow the official tutorial at <https://martinjzhang.github.io/scdrs/>. Reproduce the example results on the tutorial dataset (Tabula Muris × height GWAS). This validates your installation.

Exercise 4.3: Run scDRS on Smillie × de Lange. Using the gene-set file from Module 2 and the Smillie AnnData from Module 3:

```
scdrs compute-score \  
  --h5ad-file smillie.h5ad \  
  --h5ad-species human --gs-species human \  
  --gs-file uc_delange_top1000.gs \  
  --cov-file smillie_covariates.tsv \  
  --flag-filter-data True --flag-raw-count True \  
  --out-folder results/scdrs/smillie_delange/
```

Then run group analysis at broad granularity:

```
scdrs perform-downstream \  
  --h5ad-file smillie.h5ad \  
  --score-file results/scdrs/smillie_delange/UC.score.gz \  
  --group-analysis cell_type_broad \  
  --out-folder results/scdrs/smillie_delange_broad/
```

Inspect outputs. The group-analysis output should contain per-cell-type Z-scores and p-values. Top-ranked cell types for UC in Smillie should include immune populations (monocyte/macrophage, plasma, T cell).

Exercise 4.4: Install seismicGWAS in R.

```
install.packages("devtools")  
devtools::install_github("ylaboratory/seismicGWAS")
```

Run the seismicGWAS tutorial.

Exercise 4.5: Run seismicGWAS on Smillie × de Lange. Convert your Smillie AnnData to SingleCellExperiment using `zellkonverter` in R. Then:

```
library(seismicGWAS)  
sce <- readRDS("smillie_sce.rds")  
  
# Specificity (GWAS-independent)  
spec_broad <- calc_specificity(sce, ct_label_col = "cell_type_broad")  
  
# Load MAGMA gene Z-scores (from Module 2)  
magma_z <- read.table("step2_gene.genes.out", header = TRUE)  
  
# Regression
```

```
res_broad <- get_ct_trait_associations(spec_broad, magma_z)
print(res_broad)
```

The top cell types should be similar to your scDRS results — but not identical. Note any differences.

Exercise 4.6: Compare methods. Make a scatterplot of scDRS Z-score vs seismicGWAS regression coefficient across cell types in Smillie. The correlation should be moderate-to-high (~0.6-0.9), reflecting the shared upstream MAGMA inputs. Note any cell types that are outliers (one method ranks them high, the other doesn't) — these are the most informative cases for investigation.

Deliverable

A directory (module4_methods/) containing:

- scDRS results for Smillie × de Lange (broad and fine tier)
- seismicGWAS results for Smillie × de Lange (broad and fine tier)
- A scatterplot comparing the two methods
- A short markdown summary identifying:
 - Top 5 cell types per method
 - Cell types where the methods agree
 - Cell types where they disagree (potential investigation targets)

Self-assessment checklist

- ☐ I understand the conceptual framework of polygenic cell-type prioritization.
 - ☐ I can describe scDRS in 4 sentences.
 - ☐ I can describe seismicGWAS in 4 sentences.
 - ☐ I understand why these methods aren't statistically independent.
 - ☐ I have run both methods on Smillie × de Lange and can interpret the outputs.
-

Module 5: Cell-Type Harmonization

Learning objectives. Explain the harmonization problem and why it's hard. Use the Cell Ontology to map cell types across atlases. Apply scOntoMatch to a multi-atlas dataset. Recognize when manual mapping is necessary. Implement a marker-gene-based QC step on a harmonized vocabulary. Understand the v2-setup decisions for CL-aware harmonization.

Time estimate. 12-15 hours.

Prerequisites. Module 3.

Lecture: The harmonization problem

Different scRNA-seq atlases of the same tissue use different cell-type labels. Smillie 2019 reports "Inflammatory Monocytes"; Kong 2023 reports "Mono_CXCL10"; Menillo 2024 reports "MNP_inflam." All three labels likely refer to the same biological population, but the strings don't match. To compare cell-type rankings across atlases, you must first establish a common vocabulary.

This is the harmonization problem, and it's where multi-atlas projects spend most of their unanticipated time.

The challenge has several layers:

Naming differences. Different labs use different conventions. "TA" means "transit-amplifying" cell to some, something else to others. "DC" can mean dendritic cell, distal colon, or doctorate.

Granularity differences. Smillie may have 5 distinct T-cell subsets at fine tier; Kong may have just 2; Mennillo somewhere in between. To compare across atlases, you need to roll up to the lowest-resolution common vocabulary.

Biological differences. UC tissue contains inflammatory cell states (BEST4+ enterocytes, inflammatory fibroblasts) that healthy tissue lacks. When comparing UC atlases to HCA Gut (healthy), some cell types simply aren't present in both — the intersection rule from §2.4.3 handles this.

Subjective annotation. Two labs analyzing identical data might cluster differently and assign different labels to the same cells. The marker-gene QC step at end of M2 helps catch the worst cases.

Lecture: The Cell Ontology

The Cell Ontology (CL) is a controlled vocabulary of cell types maintained by the OBO Foundry. Each cell type has: - A unique CL identifier (e.g., CL:0000084 = T cell) - A canonical name - Synonyms - A parent-child relationship structure (T cell is_a leukocyte; leukocyte is_a hematopoietic cell)

The CL has ~3000 cell types, with substantial coverage of immune, neural, and epithelial cells.

For our v1 harmonization, the CL serves two purposes:

(1) A common vocabulary. Instead of mapping "Inflammatory Monocytes" → "Mono_CXCL10" → "MNP_inflam" pairwise, map all three to CL:0002053 (nonclassical monocyte) or CL:0000576 (monocyte) and let the CL provide the bridge.

(2) Cross-tissue extensibility for v2. Immune cells (T, B, plasma, monocyte, macrophage, DC, mast, NK) have CL parent terms that admit both gut and synovium daughters. By assigning v1's gut immune cells to these parent terms, the v2 cross-tissue analysis (UC + CD + RA) can compare T cells across gut and synovium without re-harmonizing v1.

The Osumi-Sutherland et al. 2021 paper (*Nat Cell Biol* 23:1129–1135) describes the use of CL for cross-atlas harmonization in Human Cell Atlas projects. Read it.

Lecture: scOntoMatch and the harmonization workflow

scOntoMatch (Garcia-Alonso et al., Papatheodorou Group) automates partial harmonization by matching atlas cell-type labels to CL terms via fuzzy text matching plus marker-gene similarity. It's not perfect but provides a starting point.

The standard harmonization workflow:

(1) Extract cell-type labels from each atlas. For each atlas, dump the unique values of `adata.obs['cell_type']` (or whatever column holds labels).

(2) Run scOntoMatch. Get suggested CL term mappings for each label.

(3) Manual review. Inspect each mapping. scOntoMatch will get some right, some wrong, and some partially correct. Manual review fixes them.

(4) Build the crosswalk CSV. Schema (per v2-enablement decision):

atlas	original_label	broad	fine	cl_term	cl_parent_
smillie	Inflammatory Monocytes	Monocyte/Mac	Inflammatory monocyte	CL:0002053	
kong	Mono_CXCL10	Monocyte/Mac	Inflammatory monocyte	CL:0002053	...
mennillo	MNP_inflam	Monocyte/Mac	Inflammatory monocyte	CL:0002053	..

(5) Marker-gene QC. For each broad cell type, verify canonical marker expression is consistent across atlases (per §2.6 marker-gene QC). Flag and re-map any inconsistencies.

Lecture: Sub-clustering depth handling

When atlases differ in fine-tier sub-clustering depth (Smillie has 5 T-cell subsets; Kong has 2), you can't directly compare. The v1 plan uses *roll-up to lowest-resolution atlas in any pairwise comparison*: when comparing Smillie to Kong, Smillie's 5 T-cell subsets are aggregated to Kong's 2 subsets (or a higher-resolution set common to both, if any).

The aggregation is done at the data level: cells originally labeled "CD4 Treg" in Smillie become "CD4 T" for the Smillie-vs-Kong comparison. The full Smillie fine-tier labels are retained in supplementary tables for the Smillie-only within-atlas analysis.

This rule is conservative — it sacrifices Smillie's finer resolution for cross-atlas comparability. The alternative (asymmetric vocabulary) makes intersection too small at fine tier when atlases differ substantially in depth.

Required readings

Osumi-Sutherland et al. 2021. *Cell type ontologies of the Human Cell Atlas*. Nat Cell Biol 23:1129–1135. Read fully. ~1.5 hours.

Diehl et al. 2016. *The Cell Ontology 2016: enhanced content, modularization, and ontology interoperability*. J Biomed Semantics 7:44. Read sections 1–3. ~1 hour.

scOntoMatch documentation. <https://github.com/Papatheodorou-Group/scOntoMatch> — read the README and tutorial. ~1 hour.

Cell Ontology browser. <https://www.ebi.ac.uk/ols4> — explore the CL hierarchy interactively. Look up T cell, monocyte, fibroblast, enterocyte. ~1 hour browsing.

Optional readings

Crow et al. 2018. *Predictability and proportionality in cell type comparison.* Nat Commun 9:884. Quantitative approach to cell-type matching across studies.

Exercises

Exercise 5.1: Browse the Cell Ontology. For each broad cell type in our v1 vocabulary, look up the corresponding CL term on <https://www.ebi.ac.uk/ols4> and record:
- T cell - B cell - Plasma cell - Monocyte - Macrophage - Dendritic cell - Mast cell - NK cell - ILC - Enterocyte - Goblet cell - Stem/TA cell - Fibroblast - Endothelial cell - Smooth muscle cell

For each, also record the parent CL term (to verify cross-tissue applicability for immune cells).

Exercise 5.2: Install scOntoMatch.

```
devtools::install_github("Papatheodorou-Group/scOntoMatch")
```

Exercise 5.3: Run scOntoMatch on Smillie cell-type labels. Extract the cell-type labels from `adata.obs['cell_type']` in Smillie. Apply `scOntoMatch` to map them to CL terms. Inspect the output: - How many labels were mapped automatically? - How many were ambiguous (multiple CL candidates)? - How many were unmapped?

Exercise 5.4: Manual crosswalk for 2 atlases. Build a crosswalk CSV manually for Smillie + Kong (use Mennillo if available, otherwise just two atlases is fine for this exercise). For each Smillie cell-type label, find the corresponding Kong label and assign a common broad and fine label, plus CL term IDs.

This is the actual M2 work. Time-box it to 4 hours for two atlases at broad tier; if it takes longer, you've gone too deep.

Exercise 5.5: Marker-gene QC. For each broad cell type in your crosswalk, compute the mean expression of canonical markers (from Module 3 Exercise 3.4) per atlas. Use a heatmap or barplot to visualize. Flag any cell type where one atlas's mean marker expression deviates by >2 SD from the others.

If you find flags, investigate: is the cell-type label in one atlas actually a different population? This is the marker-gene QC step from §2.6 of the v1 plan.

Deliverable

A directory (`module5_harmonization/`) containing: - A crosswalk CSV (`broad_tier_crosswalk_2_atlases.csv`) for 2+ atlases at broad tier - A marker-gene QC heatmap or barplot - A markdown summary noting any flags from QC and the resolution

Self-assessment checklist

- ☐ I understand why cross-atlas comparison requires harmonization.
- ☐ I can navigate the Cell Ontology and find appropriate CL terms.
- ☐ I have run `scOntoMatch` on a real atlas.

- ☐ I have built a manual crosswalk for 2+ atlases.
 - ☐ I have applied marker-gene QC and resolved at least one flag.
-

Module 6: Statistical Methods for the Project

Learning objectives. Compute Spearman ρ , top-k Jaccard, and Cohen's κ . Implement bootstrap confidence intervals for Spearman ρ . Understand multiple-testing correction (Bonferroni, BH-FDR). Implement Brown's method for combining correlated p-values. Understand permutation-based and analytical approaches to correlation matrix estimation. Apply donor-level leave-one-out resampling.

Time estimate. 15-18 hours.

Prerequisites. Modules 1-4. Solid Python statistics background.

Lecture: Concordance metrics

For comparing two cell-type rankings (e.g., Smillie vs Kong cell-type Z-scores), three metrics:

Spearman rank correlation (ρ). The Pearson correlation of ranks. Robust to monotonic transformations. Range: $[-1, +1]$. $\rho = 1$ means perfect rank agreement; $\rho = 0$ means no rank correlation; $\rho = -1$ means inverse rank agreement.

For our use case: ρ on Z-scores (not p-values) per the v1 plan. Computed via `scipy.stats.spearmanr`. Average-rank tie-breaking is the scipy default.

Statistical power: with $n = 10$ cell types, the standard error on ρ is ~ 0.3 . So $\rho = 0.6$ vs $\rho = 0.3$ may not be statistically distinguishable. Bootstrap CIs quantify this.

Top-k Jaccard. The Jaccard index of the top-k sets from each ranking:

$$\text{Jaccard}(\text{top-k}_A, \text{top-k}_B) = |\text{top-k}_A \cap \text{top-k}_B| / |\text{top-k}_A \cup \text{top-k}_B|$$

Range: $[0, 1]$. 1 means identical top-k; 0 means disjoint top-k.

Useful when you care more about *which* cell types rank highest than the full ranking. We use $k = 5$ and 10 at broad tier; $k = 5, 10, 20$ at fine tier.

Cohen's κ on FDR-significance. A measure of agreement on binary classifications (significant vs not). Accounts for chance agreement based on marginal frequencies.

$$\kappa = (P_{\text{observed}} - P_{\text{chance}}) / (1 - P_{\text{chance}})$$

Range: typically $[0, 1]$; can be negative if agreement is worse than chance. 0 means agreement equals chance; 1 means perfect agreement.

Caveat: κ has the well-known " κ paradox" — high observed agreement plus imbalanced marginals can produce low κ . Our v1 plan handles this with the marginal-saturation contingency: if $\geq 80\%$ of cell types pass $\text{FDR} < 0.05$ in both atlases, we report κ at $\text{FDR} < 0.01$ as the headline.

Lecture: Bootstrap confidence intervals

For Spearman ρ with small n , parametric CIs are unreliable. Bootstrap CIs are the workaround:

Procedure: 1. Given two paired vectors of cell-type Z-scores (Smillie's and Kong's). 2. Resample cell types with replacement (each bootstrap sample has the same n as the original). 3. Compute Spearman ρ on the resampled pair. 4. Repeat 1000 times. 5. The 2.5th and 97.5th percentiles of the bootstrap ρ distribution are the 95% CI (percentile method).

The percentile method is the v1 default. BCa (bias-corrected accelerated) is more accurate for skewed distributions but more complex to implement; deferred to revision if reviewers ask.

Implementation in Python:

```
import numpy as np
from scipy.stats import spearmanr

def bootstrap_spearman_ci(x, y, n_boot=1000, seed=42):
    rng = np.random.default_rng(seed)
    n = len(x)
    rhos = []
    for _ in range(n_boot):
        idx = rng.choice(n, size=n, replace=True)
        rho, _ = spearmanr(x[idx], y[idx])
        rhos.append(rho)
    rhos = np.array(rhos)
    return np.percentile(rhos, 2.5), np.percentile(rhos, 97.5)
```

Lecture: Multiple-testing correction

When you compute many tests (~60+ comparisons across the five concordance axes), some will be “significant” by chance. Multiple-testing correction adjusts for this.

Bonferroni divides the desired α (e.g., 0.05) by the number of tests. Conservative; controls family-wise error rate (FWER). For 60 tests at $\alpha = 0.05$, Bonferroni-corrected $\alpha = 0.000833$.

Benjamini-Hochberg FDR (BH-FDR) controls the *false discovery rate* — the expected proportion of false positives among declared positives. Less conservative than Bonferroni; more powerful.

Procedure: sort p-values ascending. For each rank i (1-indexed), the BH-corrected threshold is $(i / n) \times \alpha$. Declare significance for any p-value at or below its rank-specific threshold, plus all p-values below it.

The v1 plan uses BH-FDR < 0.05 across the secondary comparison battery.

```
from statsmodels.stats.multitest import multipletests
reject, p_corrected, _, _ = multipletests(p_values, alpha=0.05, method='fdr_bh')
```

Lecture: Brown's method for combining correlated p-values

When you have multiple p-values testing the same hypothesis on correlated data (e.g., per-cell-type p-values from three atlases studying UC), how do you combine them?

Fisher's method combines independent p-values:

$$\chi^2 = -2 \sum \ln(p_i)$$

Under independence, χ^2 is chi-squared distributed with $2k$ degrees of freedom (k = number of inputs). Anti-conservative when inputs are correlated (as they are for atlases sharing GWAS inputs and tissue biology).

Brown's method (Brown 1975) corrects Fisher's for known correlation between inputs. The correction approximates the test statistic distribution as a scaled chi-squared with adjusted degrees of freedom, where the adjustment depends on the correlation matrix between input p-values (or, more precisely, between their underlying Z-scores).

Kost-McDermott (2002) provides an analytical formula for the correlation correction in Brown's method, which is what `EmpiricalBrownsMethod::kotsMethod()` implements in R.

The catch: you need an estimate of the correlation matrix. Where does it come from?

For our v1, we estimate it *empirically* from per-null-draw cell-type-level test statistics generated by scDRS's internal Monte Carlo (and seismicGWAS's gene-Z permutations). For each cell type and atlas pair, we compute the Pearson correlation across 1000 null draws — that's the correlation entry that goes into Kost-McDermott.

This empirical-Kost-McDermott approach is what makes our regime 2 defensible. Without an empirical correlation matrix, the Brown's method is using a theoretical assumption that may not hold.

Lecture: Donor-level leave-one-out

To quantify uncertainty from donor sampling within atlases, we use donor-level leave-one-out (LOO):

1. Take the atlas (say, Smillie with 30 donors).
2. Remove one donor's cells.
3. Re-run scDRS on the remaining cells.
4. Recompute cell-type-level Z-scores.
5. Recompute concordance metrics with another atlas.
6. Repeat for each donor.
7. The 2.5th and 97.5th percentiles of the LOO concordance distribution are the LOO jackknife range.

This isn't a true confidence interval (the resamples are autocorrelated), but it captures variance from donor-level sampling. Reported as "95% LOO jackknife range," not "95% CI."

For Kong specifically (12 donors total), enforce a minimum-after-LOO threshold of ≥ 5 cases per group to avoid scoring on tiny samples.

Required readings

Brown 1975. *A method for combining non-independent, one-sided tests of significance.* Biometrics 31:987–992. The original paper. Short. ~1 hour.

Kost & McDermott 2002. *Combining dependent p-values.* Stat Probab Lett 60:183–190. The analytical formula. ~1 hour.

Benjamini & Hochberg 1995. *Controlling the false discovery rate.* JRSS-B 57:289–300. The original BH-FDR paper. ~1 hour.

Efron & Tibshirani 1993. *An Introduction to the Bootstrap*, Chapters 1, 6, 13. The canonical bootstrap reference. ~3 hours of reading.

Optional readings

Storey 2002. A direct approach to false discovery rates. The Storey-Tibshirani q-value. Useful background.

Davison & Hinkley 1997. *Bootstrap Methods and their Application.* For deeper bootstrap theory.

Exercises

Exercise 6.1: Implement Spearman ρ from scratch. Without using scipy, implement Spearman ρ in pure Python/numpy. Test on a small dataset. Compare to `scipy.stats.spearmanr` to verify.

Exercise 6.2: Implement bootstrap CIs. Using the function from the lecture, compute bootstrap 95% CIs for Spearman ρ on the Smillie $\times$ de Lange vs Smillie $\times$ Liu cell-type Z-score vectors (from Module 4). The CI width tells you how much sampling variance to expect at fine tier.

Exercise 6.3: Implement BH-FDR. Without using statsmodels, implement BH-FDR in pure Python. Test on a synthetic example with 100 p-values, half drawn from null and half from alternative. Verify your implementation matches `statsmodels.stats.multitest.multipletests`.

Exercise 6.4: Implement Brown’s method (Kost-McDermott analytical). Given three p-values p_1 , p_2 , p_3 and a 3×3 correlation matrix R , implement the Kost-McDermott formula in Python. Verify on a small example: when R is the identity matrix, your method should reduce to Fisher’s method.

Hint: The formula involves computing E and Var of the Fisher’s statistic under the correlation structure, then scaling. Reference: Kost-McDermott 2002 equations 4–6.

Exercise 6.5: Estimate empirical correlation matrix. Generate 1000 simulated null draws of three correlated Z-scores (use a multivariate normal with a known cor-

relation matrix). Estimate the correlation matrix from the simulations. Verify your estimate matches the truth within ~ 0.05 .

This is the procedure we'll use to estimate the cross-atlas correlation matrix in M5 of the actual project.

Exercise 6.6: Implement donor-LOO. Given a small dataset (you can simulate one), implement leave-one-donor-out resampling: for each donor, remove their data, recompute a statistic, collect across donors. Report the 2.5th and 97.5th percentiles as the LOO jackknife range.

Deliverable

A Python module (`stats_utils.py`) with: - `spearman_rho_with_ci(x, y, n_boot=1000, seed=42)` returning (ρ , lower, upper) - `bh_fdr(p_values, alpha=0.05)` returning (rejected_indices, corrected_p) - `brown_kost_mcdermott(p_values, correlation_matrix)` returning combined p-value - `donor_loo(data, donor_ids, statistic_fn)` returning (point_estimate, lower, upper)

Tested on synthetic data. This module is reused throughout the actual project.

Self-assessment checklist

- ☐ I can compute and interpret Spearman ρ , Jaccard, and κ .
 - ☐ I can implement bootstrap CIs for any statistic.
 - ☐ I understand the difference between FWER and FDR control.
 - ☐ I can implement Brown's method given a correlation matrix.
 - ☐ I have a working stats utility module ready for the project.
-

Module 7: UC Biology

Learning objectives. Describe the anatomy and cell-type composition of healthy colon mucosa. Describe the pathophysiology of ulcerative colitis. Identify the major UC-implicated genetic pathways and their cell-type contexts. Understand vedolizumab and the Mennillo time-course atlas.

Time estimate. 8-10 hours.

Prerequisites. Module 1, Module 3.

Lecture: Colon mucosa anatomy

The colon mucosa is a layered tissue with three principal compartments:

Epithelium. The single-cell-thick layer lining the colon lumen. Composed of: - *Enterocytes* (absorptive cells): the most abundant epithelial cell, responsible for water and ion absorption. - *Goblet cells*: secrete mucus, the protective barrier between epithelium and luminal microbiota. - *Stem cells and transit-amplifying (TA) cells*: in

the crypts at the base of the epithelium, continuously dividing to replace the epithelium every 3–5 days. - *BEST4+ enterocytes*: a recently characterized epithelial subset with ion-channel expression; expanded in UC. - *Tuft cells, enteroendocrine cells*: rare specialized epithelial populations.

Lamina propria. The connective tissue immediately below the epithelium. Composed of: - *Fibroblasts*: structural cells; multiple subtypes (myofibroblasts, telocytes, inflammatory fibroblasts in UC). - *Endothelial cells*: line blood and lymphatic vessels. - *Smooth muscle cells*: in the muscularis mucosa and around vessels. - *Resident immune cells*: T cells, B cells, plasma cells, macrophages, dendritic cells, mast cells, ILCs — all present in healthy mucosa as part of normal gut immunity.

Submucosa and muscularis externa. Deeper layers, less commonly captured in scRNA-seq of colon mucosa.

Lecture: UC pathophysiology

Ulcerative colitis is a chronic inflammatory disease of the colon mucosa. Key features:

Continuous inflammation. Unlike Crohn's disease (which has skip lesions), UC inflammation is continuous, starting from the rectum and extending proximally.

Mucosal restricted. UC inflammation is largely restricted to the mucosa (epithelium + lamina propria), unlike Crohn's which can be transmural.

Cell-type expansion. UC tissue has expanded inflammatory cell populations: - *Inflammatory monocytes/macrophages* infiltrate the lamina propria. - *Plasma cells* (especially IgG-producing) increase dramatically. - *Inflammatory fibroblasts* with COL1A1+, IL11+ expression appear; not present (or very rare) in healthy tissue. - *BEST4+ enterocytes* expand, with possible role in disease. - *Th17 and Treg cells* shift in balance.

Tissue damage. Ulceration of the epithelium, crypt distortion, goblet cell depletion.

Lecture: UC genetics

UC has heritability ~0.3 (genetic factors explain ~30% of risk). The genetic landscape implicates:

IL-23 / Th17 pathway. *IL23R* is the strongest non-MHC UC GWAS signal. IL-23 activates Th17 cells; Th17 cells produce IL-17 which recruits neutrophils to mucosa. Pathway genes: *IL23R, IL12B, JAK2, TYK2, STAT3, RORC*.

Type 1 interferon response. *TYK2* (which is in the IL-23 pathway but also in type 1 IFN signaling), *STAT1*, others.

Barrier function. Genes affecting epithelial barrier integrity: *MUC1, HNF4A*.

Th2 / type 2 immunity. Less prominent than Th17, but present: *IL13, GATA3*.

Innate immunity. *NOD2* (more strongly associated with CD than UC), pattern recognition receptors.

MHC. As discussed in Module 1, the strongest signal but excluded from our gene set.

The cell-type prioritization question: when scDRS or seismicGWAS scores cell types in colon mucosa against UC GWAS, the expected top hits are immune cells expressing IL-23-pathway genes (T cells, monocyte/macrophages) and possibly inflammatory fibroblasts and BEST4+ enterocytes (where pathway genes converge in disease).

Lecture: Anti-integrin therapy and the Mennillo time-course atlas

Vedolizumab is an anti- $\alpha 4\beta 7$ integrin antibody approved for UC. It blocks gut-homing T cells from migrating into intestinal mucosa, reducing tissue inflammation.

The Mennillo et al. 2024 atlas profiles UC patients before and after vedolizumab treatment. Pre-treatment cells reflect active UC; post-treatment cells reflect the response to therapy. The atlas is interesting biologically because it tracks how cell-type composition and gene expression change with therapy.

For our v1, we use *pre-treatment baseline samples only* (per the §2.3 Mennillo time-point policy) — these are most comparable to Smillie 2019 and Kong 2023 active-UC samples. Post-treatment samples are out of scope for the v1 cross-atlas comparison but interesting for v2 follow-up.

Required readings

Smillie et al. 2019 (re-read). Focus on the Discussion and the cell-type findings — what populations are expanded in UC vs healthy?

Kong et al. 2023. *The landscape of immune dysregulation in Crohn's disease revealed through single-cell transcriptomic profiling in the ileum and colon.* Immunity 56:444-458. Read fully — even though we use only the UC arm. ~2 hours.

Mennillo et al. 2024. *Single-cell and spatial multi-omics highlight effects of anti-integrin therapy across cellular compartments in ulcerative colitis.* Nat Commun 15:1493. Read Abstract, Introduction, Results focusing on cell-type composition. ~2 hours.

Optional readings

Graham & Xavier 2020. *Pathway paradigms revealed from the genetics of inflammatory bowel disease.* Nature 578:527-539. Synthesizes IBD genetics with pathway biology.

McGovern et al. 2015. *Genetics of Inflammatory Bowel Diseases.* Gastroenterology 149:1163-1176. Useful overview of IBD genetics.

Exercises

Exercise 7.1: Cell-type cheat sheet. Create a one-page reference document with all 14 broad cell types in our gut mucosa vocabulary. For each, list: - Canonical markers (3-5 genes) - Anatomical location in the mucosa - Expected change in UC (expanded, depleted, qualitatively different, no change) - Known GWAS-implicated genes preferentially expressed in this cell type (where applicable)

This is your reference document for interpreting cell-type prioritization results.

Exercise 7.2: UC gene list. Compile a list of 20–30 UC GWAS-implicated genes from de Lange 2017 and Liu 2023. For each, identify the canonical cell-type expression context (which cell type in colon mucosa expresses this gene most). Use Smillie 2019 supplementary tables or scanpy to look up.

Exercise 7.3: Predict cell-type rankings. Before running any analysis, predict which cell types you expect to rank highest in scDRS for UC. Write down your predictions with reasoning. After running scDRS in M3, compare your predictions to the actual results. This exercise calibrates your biological intuition.

Deliverable

A directory (module7_uc_biology/) containing: - A one-page cell-type cheat sheet (PDF or markdown) - A UC-gene-to-cell-type table - A pre-analysis prediction document

Self-assessment checklist

- ☐ I can describe colon mucosa anatomy.
 - ☐ I can describe UC pathophysiology in 3 sentences.
 - ☐ I know the major UC GWAS-implicated pathways and their cell-type contexts.
 - ☐ I have a cell-type cheat sheet ready for interpreting results.
-

Module 8: Atlas Integration and v2 Horizon

Learning objectives. Understand atlas integration methods (scVI, scANVI, Harmony, Seurat) at a conceptual level. Recognize when integration is necessary and when it isn't. Understand why integration is difficult cross-tissue. Know what v2 (UC + CD + RA cross-tissue generalization) will require methodologically.

Time estimate. 8–10 hours.

Prerequisites. Modules 3, 5.

Note. This module is *optional* for v1 execution — regime 3 (integration) is deferred to v2. But understanding integration helps interpret Pan-GI results in v1 (Pan-GI is an scVI-integrated atlas), and prepares you for v2 work.

Lecture: The integration problem

When you combine scRNA-seq data from multiple studies, batch effects emerge. “Batch” can mean: different labs, different 10x kits, different sequencing runs, different sample-processing protocols, different days. Batch effects can dominate biological signal — cells from the same biological cell type but different batches may cluster separately, while cells from different biological cell types in the same batch may cluster together.

Atlas integration aligns cells across batches such that biological similarity (same cell type) drives clustering rather than batch identity.

Lecture: Integration methods

Modern integration methods broadly split into:

Linear methods. PCA-based, mutual nearest neighbors (MNN, Haghverdi et al.). Fast, simple, but limited expressiveness.

Nonlinear methods. Deep learning-based: scVI (Lopez et al. 2018), scANVI (semi-supervised version of scVI). These learn a low-dimensional latent representation that aligns batches. More flexible but require more compute and tuning.

Other methods. Harmony (Korsunsky et al. 2019) uses iterative correction; Seurat's CCA-based integration uses canonical correlation analysis.

For our v2, scANVI is the method of choice because: 1. It accepts cell-type labels as supervision (semi-supervised), useful when partial labels are available. 2. It produces a generative model that can be used for transfer learning. 3. It has good documented performance on cross-study integration in immune tissues.

Lecture: Integration QC

After integration, you need to verify that the integration “worked” — biological signal preserved, batch effects removed. Standard metrics:

Adjusted Rand Index (ARI) measures cluster agreement with biological labels (high = good).

Average Silhouette Width (ASW) measures cluster compactness (high = good).

kBET (k-nearest-neighbor batch effect test) measures whether cells of the same batch are over-represented in their kNN neighborhood (high p-value = good integration).

Local Inverse Simpson's Index (LISI) quantifies batch mixing in local neighborhoods.

For v2 cross-tissue integration (gut + synovium), all of these will be harder to satisfy because batch effects (study) are confounded with biology (tissue).

Lecture: Why cross-tissue integration is hard

Within-disease, within-tissue integration (UC + CD in gut) is methodologically routine — the cell-type taxonomy is shared, batch effects are technical, and integration methods reliably align cells.

Cross-tissue integration (gut + synovium) is fundamentally harder. Tissues have different cell types: gut has enterocytes; synovium has synoviocytes. Some cell types are present in both (T cells, macrophages) but their gene expression differs because tissue context differs (gut macrophages express ALOX15, IL10; synovial macrophages express MERTK, IL1B differently). The “alignment” that integration tries to achieve

is unclear: should T cells from gut and synovium be aligned (they're the same lineage but different tissue contexts)?

The v2 plan handles this by: 1. *Tier 1 (cross-tissue)*: shared immune lineages, aligned via scANVI with disease + tissue + study covariates. 2. *Tier 2 (within-tissue, broad)*: tissue-specific broad cell types, no cross-tissue integration. 3. *Tier 3 (within-tissue, fine)*: fine cell states within each tissue, no cross-tissue integration.

The v2 paper's headline finding will likely be that cross-tissue integration *partially* works (immune lineages align reasonably) but *fails* for tissue-specific cell types — which is itself a methodologically interesting finding.

Required readings (optional for v1)

Lopez et al. 2018. *Deep generative modeling for single-cell transcriptomics*. Nat Methods 15:1053–1058. The scVI paper. ~2 hours.

Luecken et al. 2022. *Benchmarking atlas-level data integration in single-cell genomics*. Nat Methods 19:41–50. The integration benchmark. Useful for understanding which methods work when. ~2 hours.

Optional readings

Zhang et al. 2019 AMP-RA Phase 1 — for v2 RA atlas familiarity.

Zhang et al. 2023 AMP-RA Phase 2 — same.

Exercises (optional)

Exercise 8.1: Run scVI on a small dataset. Follow the scvi-tools tutorial at <https://docs.scvi-tools.org/>. Run integration on a small example dataset. ~3 hours.

Exercise 8.2: Compute integration QC. On the integrated output from 8.1, compute ARI, ASW, and (optionally) kBET. Compare to the unintegrated baseline.

Deliverable (optional)

Notes from running the scVI tutorial. No formal deliverable required; this module is for v2 preparation.

Self-assessment checklist

- ☐ I understand why batch effects arise in multi-study scRNA-seq.
 - ☐ I know what scVI / scANVI do at a conceptual level.
 - ☐ I understand why cross-tissue integration is harder than within-tissue.
 - ☐ I have a sense of what v2 will require methodologically.
-

Module 9: Reproducibility and Project Management

Learning objectives. Set up a reproducible Python and R environment. Use git/GitHub effectively for a multi-author project. Understand DECISIONS.md and pre-registration. Set up Hummingbird (UCSC's HPC cluster) for scDRS runs. Manage long-running analyses with bash scripts and SLURM job submission.

Time estimate. 8-10 hours.

Prerequisites. Modules 1-4.

Lecture: Why reproducibility matters

A year-long, two-author, multi-stage analysis project has many opportunities to fail: - Author A runs `scdrs compute-score` with one set of arguments; Author B reproduces with slightly different arguments. Results differ. You can't tell which is "right." - A package version updates mid-project. Old results no longer reproduce. - A bug is discovered. You need to re-run earlier steps to get correct results, but the original scripts are lost or modified. - A reviewer asks for a sensitivity analysis. You can't easily re-run the relevant step because it depends on a buried Jupyter notebook on someone's laptop.

Reproducibility is the discipline that prevents these failures. Practical reproducibility for our v1:

Version-pinned environments. `pyproject.toml` (or `requirements.txt` + lockfile) for Python; `renv.lock` for R. Docker image for the entire stack if possible.

Version-pinned data. HGNC release date, Cell Ontology release, atlas download dates, GWAS download dates — all in DECISIONS.md.

Random seeds. Every random procedure has a fixed seed: scDRS Monte Carlo seed, bootstrap seed, donor-LOO ordering seed. The v1 plan specifies `seed=42` for the headline.

Append-only DECISIONS.md. Every analytical commitment is recorded with a date. Deviations require new dated entries explaining the reason.

Git-tracked code with no untracked secrets. Everything in version control. No hardcoded paths, no untracked credential files.

Notebooks as exploration; scripts as production. Jupyter notebooks for exploration; refactored Python scripts (or Snakemake/Nextflow pipelines) for the actual analysis runs that produce headline numbers.

Lecture: Git and GitHub for two-author projects

The two-author dynamic adds complexity to git workflows:

Branch per major task. Each author works on a feature branch for non-trivial work. PRs (pull requests) with review before merging to main.

Frequent, descriptive commits. “Fixed bug” is not a useful commit message. “Fixed harmonization crosswalk for Smillie monocyte subpopulations after marker-gene QC flagged inconsistency” is.

Clear ownership. Each module of code has a primary maintainer (per the README division of labor). Modifications by the other author go through PR.

Tagged releases at milestones. v1-M1-complete, v1-M3-gate1-passed, etc. Tags make it easy to revisit a specific project state.

Lecture: Hummingbird (UCSC HPC) basics

UCSC’s research computing cluster is Hummingbird. Key things to know:

SSH access. `ssh username@hb.ucsc.edu`. Set up SSH keys for password-less login.

SLURM job submission. Hummingbird uses SLURM. Submit jobs with `sbatch script.sh`. Check status with `queue -u username`. Cancel with `scancel jobid`.

Storage. Home directory has limited quota. Project directories on `/hb/projects/` or scratch space have larger quotas. Plan where atlas files (multi-GB) will live.

Memory and CPU requests. Each SLURM job specifies its memory and CPU requirements. `scDRS` runs need ~15-20 GB for typical atlases, ~30 GB for Pan-GI. Request appropriately.

Modules. Software is loaded via `module load python/3.10` etc. Some packages may need to be installed in user space (conda environments).

A typical SLURM script for a `scDRS` run:

```
#!/bin/bash
#SBATCH --job-name=scdrs-smillie-delange
#SBATCH --time=2:00:00
#SBATCH --mem=20G
#SBATCH --cpus-per-task=4
#SBATCH --output=logs/scdrs-smillie-delange-%j.out

module load python/3.10
source ~/envs/uc-scrna/bin/activate

scdrs compute-score \
  --h5ad-file data/atlases/smillie.h5ad \
  --gs-file data/gwas/uc_delange_top1000.gs \
  --cov-file data/atlases/smillie_covariates.tsv \
  --out-folder results/scdrs/smillie_delange/
```

Lecture: Project management for a 12-month analysis

A 12-month, two-author project has known failure modes. Mitigations from the v1 plan:

Three explicit gates (M3, M5, M7). At each gate, you assess whether prerequisites are met before proceeding. Gates prevent “we’ll fix that later” from becoming “we never fixed that.”

Monthly PI mentor checkpoints. Bring concrete deliverables (figures, tables, code commits) to each meeting. Don’t schedule meetings for status updates; schedule them for decisions.

Write early. The introduction is drafted in M4, methods in M5, full draft by M6. If you start writing at M9, you’re behind.

Descope ladders. If you slip at a gate, you drop something — that’s the deal. The plan specifies what to drop and in what order. Adhering to the descope ladder is the discipline that prevents the project from inflating into infeasibility.

Required readings

The v1 plan itself. Read it end-to-end. Identify any sections you don’t fully understand; bring questions to your PI mentor before M0. ~3 hours.

The DECISIONS.md template (in the v1 plan). Understand what gets committed and why. ~30 minutes.

Wilson et al. 2017. *Good enough practices in scientific computing.* PLOS Comp Biol 13:e1005510. Practical reproducibility recommendations. ~1 hour.

Optional readings

Munafò et al. 2017. *A manifesto for reproducible science.* Nat Hum Behav 1:0021. Good background on the broader reproducibility crisis in science.

Exercises

Exercise 9.1: Set up a project repo. Create a fresh git repository for the v1 project (or a sandbox version). Structure:

```
uc-cross-atlas-v1/
├── README.md
├── DECISIONS.md
├── data/                # gitignored (large files)
├── code/
│   ├── 01_magma/
│   ├── 02_atlas_prep/
│   ├── 03_scdrs/
│   ├── 04_seismic/
│   ├── 05_sanity/
│   ├── 06_concordance/
│   ├── 07_regime2/
│   ├── 08_cross_method/
│   ├── 09_cross_gwas/
│   └── 10_broad_atlas/
```

```

|   |   | 11_positive_negative_controls/
|   |   | 12_figures/
|--- results/           # gitignored (large outputs)
|--- figures/
|--- notebooks/
|--- manuscript/
|--- pyproject.toml
|--- renv.lock
|--- Dockerfile

```

Push to a private GitHub repo. Add Muskaan and PI mentor as collaborators.

Exercise 9.2: Set up Hummingbird access. Get an account, set up SSH keys, try a basic sbatch job (an empty bash script that just echo "hello"). Verify your environment.

Exercise 9.3: Reproducible “hello world” pipeline. Write a SLURM script that runs the scDRS tutorial from Module 4 on Hummingbird. Verify the output matches what you got locally.

Exercise 9.4: Initial DECISIONS.md. Using the template from the v1 plan, draft an initial DECISIONS.md for your project. Fill in: - Project name, scope, dates - GWAS, atlases, methods, regimes, granularity - Statistical commitments (Z-scores, bootstrap CIs, BH-FDR, etc.) - Compute budget - Stretch ladder - Authorship plan - (Leave the v2 trajectory commitments blank if not yet PI-confirmed)

Commit to your project repo.

Deliverable

A working project repository structure with: - README.md describing the project - DECISIONS.md populated with v1 commitments - Hummingbird scDRS tutorial run successfully via SLURM - pyproject.toml and renv.lock lock files

Self-assessment checklist

- ☐ My project repo is set up with the right structure.
- ☐ Hummingbird is configured and I’ve run a SLURM job.
- ☐ I have a working DECISIONS.md.
- ☐ I have version-pinned environments for Python and R.
- ☐ I understand the gate / descope ladder discipline.

Module 10: Project Orientation and M0 Readiness

Learning objectives. Demonstrate competency across Modules 1–9. Identify any remaining knowledge gaps before M0. Prepare specific questions for the PI mentor. Lock in the M0 commitments in DECISIONS.md.

Time estimate. 6–8 hours.

Prerequisites. Modules 1–9.

Lecture: What “ready for M0” means

M0 is the planning lock-in date. Before M0, you should be able to:

1. **Explain the project scope to a peer** in 5 minutes without notes. Including the disease, atlases, methods, regimes, granularity, and what the headline claim will look like.
2. **Run scDRS and seismicGWAS end-to-end** on a tutorial dataset, interpret outputs, and debug common failures.
3. **Build a manual cell-type crosswalk** for two atlases at broad tier in 2–4 hours, including marker-gene QC.
4. **Implement Brown’s method** with empirical correlation matrix estimation. Implement bootstrap CIs for Spearman ρ .
5. **Manage a git/GitHub project** with branches, PRs, and tagged releases.
6. **Submit SLURM jobs** to Hummingbird and monitor their progress.
7. **Read the v1 plan** without confusion about technical terminology.
8. **Identify methodological choices** that affect results (e.g., MHC handling, all-cells vs UC-only) and articulate why each commitment was made.

If any of these feels uncertain, more practice is needed before M0.

Comprehensive self-assessment

Work through this checklist. For each item, rate yourself: 1 (don’t understand), 2 (mostly understand), 3 (can explain to a peer), 4 (can teach to a peer).

GWAS and MAGMA: - [] What a GWAS measures - [] Linkage disequilibrium and ancestry - [] MAGMA SNP-to-gene + gene-based test - [] MHC region and why it’s excluded - [] λ_{GC} sanity checks - [] LD reference panel matching

scRNA-seq: - [] AnnData object structure - [] Normalization approaches - [] QC metrics - [] Doublet and ambient RNA artifacts - [] Cell-type annotation

Cell-type prioritization methods: - [] scDRS algorithm and output - [] seismicGWAS algorithm and output - [] Why methods aren’t statistically independent - [] Atlas-specific Monte Carlo null - [] Why we use Z-scores (not p-values) for cross-atlas concordance

Cell-type harmonization: - [] Cell Ontology and CL terms - [] scOntoMatch + manual + marker-gene workflow - [] Sub-clustering depth roll-up rule - [] Marker-gene QC procedure

Statistical methods: - [] Spearman ρ , Jaccard, Cohen’s κ - [] Bootstrap confidence intervals (percentile method) - [] Bonferroni vs BH-FDR - [] Brown’s method with empirical correlation matrix - [] Donor-LOO uncertainty intervals

UC biology: - [] Colon mucosa anatomy and cell types - [] UC pathophysiology - [] UC GWAS implicated pathways - [] Vedolizumab and Mennillo time-course

Project management: - [] Git/GitHub two-author workflows - [] Hummingbird SLURM job submission - [] DECISIONS.md and pre-registration discipline - [] Gate / descope ladder logic

For any item rated 1 or 2, return to the relevant module's exercises before M0.

Pre-M0 questions for PI mentor

Bring these to your last pre-M0 mentor meeting:

1. **PI commitment to v2 trajectory.** Are you committed to the UC + CD + RA cross-tissue follow-up paper (18–24 months after v1 ships)? If yes, the three v2-setup decisions in v1 are locked. If no, we drop them.
2. **Authorship plan.** First-author / co-first / author order. Document in README.
3. **Hummingbird allocation.** How many node-hours are available? Is the Pan-GI memory requirement (~30 GB) feasible?
4. **Mennillo accession.** Confirm the GEO accession; verify pre-treatment $N \geq 8$.
5. **Liu 2023 N column.** Confirm per-SNP N column availability or document the fixed-N approximation.
6. **Hard deadlines.** Do you (PI) have any external deadlines that affect our plan (grant submissions, conference deadlines, your own faculty timeline)?
7. **Crisis protocol.** If we hit Gate 1 fail at M3, what's the response — drop scope, push on, escalate?

Final exercises before M0

Exercise 10.1: 5-minute project pitch. Write a 5-minute talk explaining the project to a peer. Practice with someone who isn't familiar with the project. Get feedback on clarity. Revise.

Exercise 10.2: Methods text draft (placeholder). Draft a 1-page Methods section using the v1 plan's commitments. Don't worry about formatting; just write the prose that would go into a paper. This forces you to articulate the choices.

Exercise 10.3: Risk register review. Read the v1 plan's Risk Register. For each risk, assess: do you understand the mitigation? Have you done what's needed (e.g., Hummingbird allocation verified, fallback atlases identified)?

Exercise 10.4: Final checklist. Confirm: - [] All 10 modules' deliverables produced - [] DECISIONS.md drafted - [] Hummingbird access working - [] Project repo set up with README, DECISIONS.md, pyproject.toml, renv.lock - [] PI mentor questions prepared - [] You can run scDRS + seismicGWAS on Smillie × de Lange end-to-end - [] Crosswalk for 2+ atlases at broad tier built

Deliverable

A “Pre-M0 readiness packet” containing: - Self-assessment scores - 5-minute pitch (written or recorded) - Methods text placeholder (1 page) - DECISIONS.md draft - List of questions for PI mentor

Bring all of this to your final pre-M0 meeting.

Self-assessment checklist

- ☐ All 9 prior modules’ deliverables completed.
- ☐ All self-assessment items rated 3+ (can explain to a peer).
- ☐ Project repo and Hummingbird ready.
- ☐ PI mentor questions prepared.
- ☐ Methods text placeholder drafted.

If you can check all these boxes, you’re ready for M0.

Course Conclusion

You’ve now completed ~100-150 hours of focused preparation across 10 modules. You should have:

1. A working understanding of GWAS, MAGMA, scRNA-seq, scDRS, seismicGWAS, cell-type harmonization, the project’s statistical methods, and UC biology.
2. A working project repository with reproducibility scaffolding.
3. Hummingbird access for HPC compute.
4. A drafted DECISIONS.md with v1 commitments.
5. Concrete artifacts (covariate files, crosswalks, MAGMA outputs) that feed directly into M1 of the actual project.

What this course does NOT cover: - Specific paper-writing technique (separate skill, learned through practice) - v2 detailed methodology (that’s for M9-M12 of v1, when you have bandwidth) - Specific cluster admin tasks at UCSC (your PI or an HPC consultant covers these) - Wet-lab biology beyond what’s needed to interpret scRNA-seq atlases

Maintenance during the project. This curriculum’s deliverables become the M1 starting points. Don’t archive them; integrate them into the project repo. Some modules (especially Module 6 stats utilities) you’ll keep using through M11.

If you fall behind during the project, return to the relevant module’s exercises. The exercises are calibrated to the actual project tasks.

Ship it. The course prepares you. The project ships through execution discipline. Use the year you have.